OCTOBER 24, 2026 · CA’ FOSCARI, MESTRE



API Archaeology

Working with APIs you don’t understand *(yet)*

Matteo Valerio · Full-stack developer

```
GET /api/v2/records/48213  200 OK
{
  "id": 48213,
  "code": "A-42",
  "flg_att": "S",
  "status": 7,
  "amount": "1250,00",
  "updated": "2019-03-14",
  "created": 1552521600,
  "notes": null
}
```

What does this API do?

No documentation. No tests. Nobody left who wrote it.

Take ten seconds.



WHO'S DIGGING

Hi, I'm Matteo Valerio

Air srl · Connected mobility

Full-stack developer

5+ years with React, TypeScript, Next.js and .NET. Now Node, NestJS and AWS serverless.

Ex PM, analyst and team lead

I've run onboarding from both sides of the table, and seen it rushed on both.

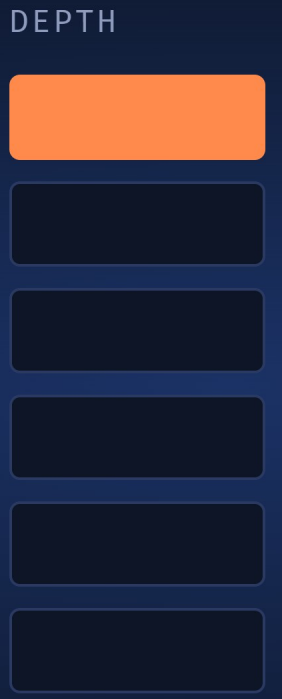
Ex classical violinist

Years spent reading old scores. Now I read old code.

LAYER 1 OF 6

The dig site

Why most of our work happens in code we didn’t write





DAY ONE

Welcome aboard! Here’s the repo.

```
$ git clone git@internal:core/backend.git
$ cat README.md
  # backend
  TODO: write documentation
$ git log -1 --format=%cr -- README.md
  6 years ago
```

New job or new client: access arrives fast, context doesn’t.



THE TEMPTATION

Why not just rewrite it?

No budget

The business pays for features and stability. A rewrite delivers neither for months.

No downtime

The system keeps serving users while you learn it. You change the engine mid-flight.

Hidden knowledge

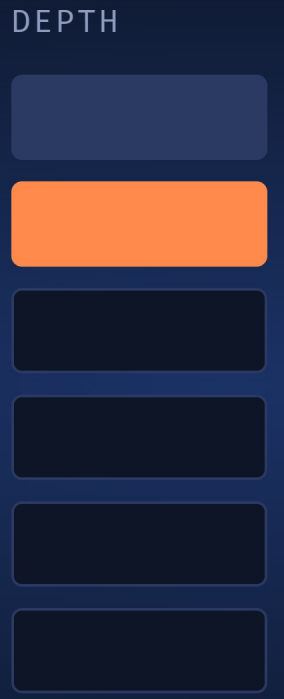
Years of edge cases live in that code, and in nobody's head anymore.



LAYER 2 OF 6

Survey the surface

Map the site before you dig: use cases, flows, unknowns





METHOD

Dig in layers, not in holes

01 · SURFACE	Use cases: what does it do for its users?	-1 m
02 · FLOWS	How does one request travel through it?	-2 m
03 · MODULES	Who owns which data and which rules?	-3 m
04 · DETAILS	Why is this line written this way?	-4 m

Go one layer deeper only when the layer above makes sense.



SURFACE

Start from what the business uses

Goal: name the 3 to 5 use cases that keep the business running.

Production traffic

Logs and metrics show which endpoints run, how often and for whom.

The consumers

Frontends, apps, partner integrations, scripts: their calls are the real spec.

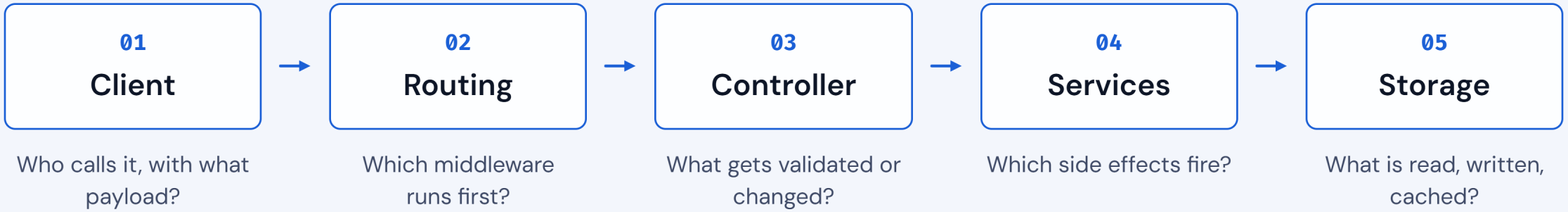
The people

Support, product, the longest-serving dev: ask what breaks and what users complain about.



SURFACE → FLOWS

Trace one flow, end to end



Write every hop down: that sketch is the start of your documentation.

THE TURNING POINT

Wait for the click

BEFORE

Knowing what the flow is

You can name every step and draw every box on the whiteboard.

→

AFTER THE CLICK

Knowing the flow

You can predict what breaks when it changes, and why it was built this way.

That’s when you’re ready for the details.



LAYER 3 OF 6

The toolkit

Simple tools, used deliberately





TOOLKIT

Five tools, one goal

FIELD SKETCHES

Pen & paper

Draw the flow by hand. Going slow forces understanding.

THE ARCHIVE

Browser & templates

Date the site from its dependencies and its infrastructure files.

FIELD PARTNER

AI tools

Map the surface fast. Ask the questions you'd never ask a colleague.

WATCH THE SITE

Observation

Logs, traffic and debuggers show what really runs.

CARBON DATING

Tests

Prove what you think you know about the system.

Start analog: the click usually happens on paper, not on screen.



Dependencies tell the site's age

```
package.json (example)

"dependencies": {
  "express": "^4.13.4",
  "request": "^2.81.0", ← deprecated
  "bluebird": "^3.4.7",
  "moment": "^2.17.1", ← maintenance mode
  "auth-lib": "git+ssh://…" ← owner?
}
```

Deprecated? Find out what replaced it, and why.

Read the docs for the version you have, not the latest.

Old open issues often explain today's "features".

Internal packages: find the repo, the owner, the last commit.



Infrastructure is documentation

```
template.yaml (example)

NightlyBridgeFn:
  Properties:
    Timeout: 900 ← a long job
    Events:
      Schedule: cron(0 2 * * ? *) ← at 02:00
    Environment:
      PARTNER_ENDPOINT: !Ref PartnerUrl
      LEGACY_TABLE: !Ref RecordsLegacy ← ?
```

It answers two questions nobody could: what exists, and what is wired to what.

Schedules and timeouts describe work you never see in daytime logs.

Environment variables name the systems you depend on.

Nobody mentioned this ran at 02:00. The template did.



TOOLKIT · WATCH THE SITE

Real usage beats reading code

Logs

Filter production logs by endpoint: real payloads, errors, frequencies.

Traffic captures

Record requests with a proxy or a HAR file, then replay them in staging.

Debugger

Step through one real request and watch what changes along the way.

Query logs

Log the queries one call fires: you'll see what it really touches.



The first query you run

```
one aggregator query, any dialect

fields @timestamp, @message
| filter @message like /httpMethod/
| parse @message '"path":"*"' as path
| parse @message '"httpMethod":"*"' as method
| stats count(*) as calls,
        max(@timestamp) as last_seen
    by method, path
| sort calls desc
```

Which endpoints are alive, and how often.

Which are dead, and since when.

The real shape of the requests, not the documented one.

A baseline you can run again after your change.

Run it before you open the repository: thirty seconds of log query beats two days of reading code.



TOOLKIT · CARBON DATING

Tests that describe what it does

orders.characterization.test.js

```
describe('POST /orders: current behavior', () ⇒ {
  it('returns 200 even when stock is 0', async () ⇒ {
    const res = await request(app)
      .post('/orders')
      .send({ sku: 'A-42', qty: 3 });

    expect(res.status).toBe(200); // surprising, but real
    expect(res.body.status).toBe('PENDING');
  });
});
```

Characterization tests pin down what the system does today, quirks included.

A failing test means your mental model was wrong. Good: you just learned something.

The term comes from Michael Feathers, *Working Effectively with Legacy Code*.



LAYER 4 OF 6

Handle with care

Changing code you only partly understand





RISK

Never fix an island



Legacy code hides its side effects one hop away. Trace the whole flow first.

RISK

Five questions before any change

01 **Who consumes this?**

Frontends, other services, scripts, that forgotten cron job.

02 **What does it return today, exactly?**

Fields, order, nulls, status codes, error shapes.

03 **What else reads or writes this data?**

Other endpoints, batch jobs, reports, triggers.

04 **Is today's behavior covered by a test?**

If not, write a characterization test first.

05 **How do I roll back?**

A feature flag, a revert plan, a way to undo data changes.



RISK

Find the consumers nobody remembers

Name them from auth

API key, client id, token claims: the closest thing to a caller's name you have.

Count usage per field

Half the payload is always null. The other half is somebody's contract.

Log before you change

Ship a warning on the field you want to touch, then wait a month and read it.

Measure the gap

Traffic you cannot attribute to anyone is your real risk budget. Know the number.



EVOLVE SAFELY

Add structure, keep the contract

Document what is

An OpenAPI spec of today's behavior, quirks included.

Change additively

New fields and endpoints are safe. Renames and removals are not.

Version when you must break

/v2 beside /v1, a sunset date, and a list of who still calls v1.

Test the contract

Contract tests for the consumers you know. Logs for the ones you don't.



LAYER 5 OF 6

Grow the new around the old

Partial modernization with the Strangler Fig pattern





PATTERN

The Strangler Fig pattern



IN NATURE

A fig seed sprouts high in a host tree. Its roots grow down around the trunk until, years later, the fig stands on its own.

IN SOFTWARE

Put a facade in front of the legacy system. Move one slice of traffic at a time to new code. Retire old parts when nothing calls them.

Described by Martin Fowler in 2004, after the strangler figs he saw in Australian rainforests.
Photo: Vinayaraj, CC BY-SA 4.0, via Wikimedia Commons.

PATTERN

Intercept, coexist, retire

STEP 1

Intercept



Put a facade in front: a gateway, proxy or router. All traffic still reaches legacy.

STEP 2

Coexist



Move one flow at a time to new code. Compare responses before switching.

STEP 3

Retire



When no traffic reaches a legacy route, delete it. Repeat.

LAYER 6 OF 6

Field report

A real dig: a giant Express backend on Node 6





CASE STUDY · THE SITE

A giant Express backend on Node 6

~15	~100k	1 year	2019
endpoints migrated to Lambda	lines of JavaScript	in production on Lambda	Node 6 end of life

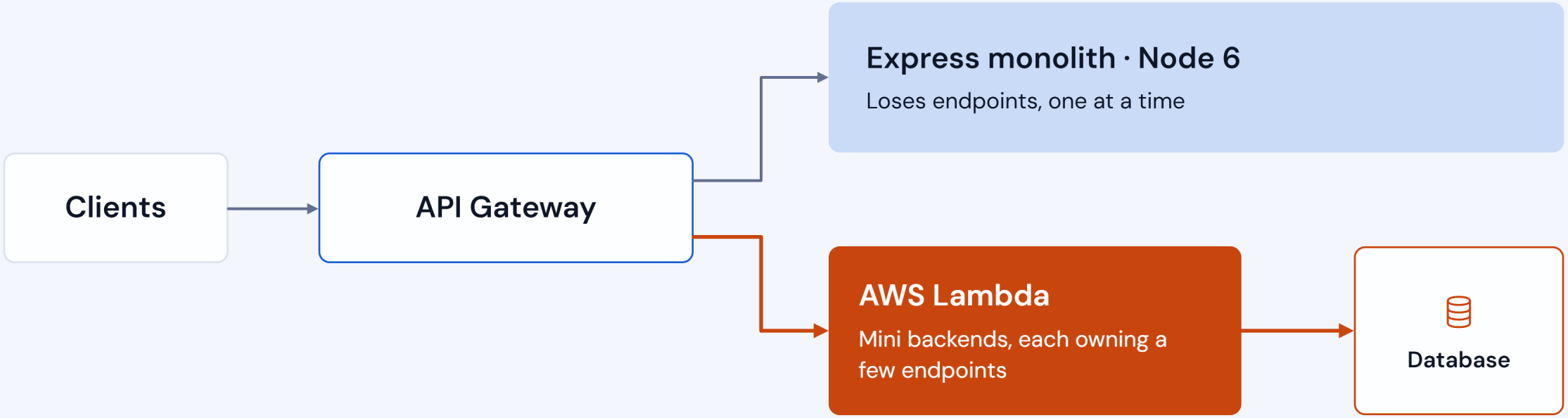
What it does: it owns a slice of master data that most of the platform reads.

- No budget for a rewrite
- Business-critical, always on
- AWS microservice platform already in place



CASE STUDY · THE MOVE

Strangle with what's already there



API Gateway sends a slice of endpoints to Lambdas; the rest still hits the monolith. The Lambdas use a database that the rest of the monolith never reads.

~100k lines of monolith. ~15k lines of Lambdas doing the same work.

CASE STUDY · FINDINGS

What the dig taught us

01

Reuse the platform you have

The mini backends ran on the company's existing AWS setup: no new platform to build.

02

Start where the data is isolated

No other part of the monolith read that database: a safe slice to move.

03

Cleanups change the contract

Remember flg_att from the first slide? I turned "S"/"N" into booleans. Another backend copying that data expected "S"/"N" or 0.



RECAP

Your field guide

01

Survey the surface first

02

Trace flows, park the unknowns

03

Wait for the click, then dig

04

Prove it with tests

05

Never fix an island

06

Grow the new around the old

ONE LAST THING

Every legacy system was someone’s greenfield project.

Dig with the care you’d want for yours.



THANK YOU

Questions? Let's dig.

SLIDES ON GITHUB

github.com/matteovalerio/api-archaeology

CONNECT ON LINKEDIN

linkedin.com/in/matteo-valerio-dev

RATE THIS TALK

Scan to leave your feedback